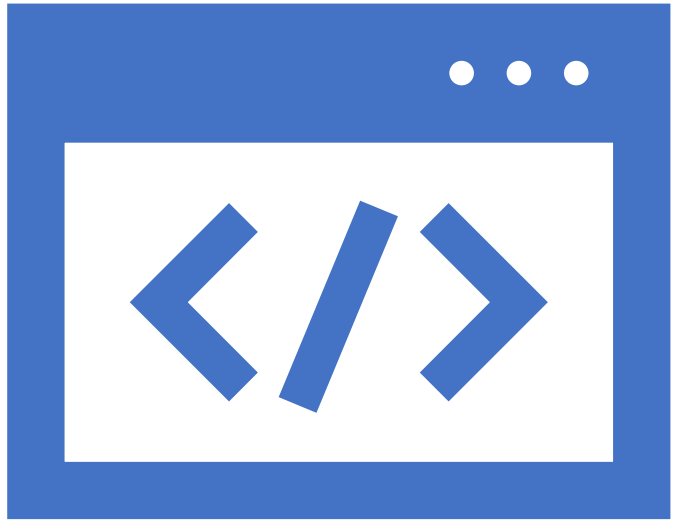
CAPSTONE PROJECT

BY Cindy, Duc, Kevin

OUR GOAL

Our goal was to transform a heterogeneous corpus into a usable system. The scope stayed practical: document upload, browsing, selection, and AI-based interactions that are grounded in the chosen corpus:

- Upload and organize files instead of leaving them as a raw folder.
- Select which documents are active for AI interactions.
- Support chat, flashcards, quiz, and code review workflows.



ARCHITECTURE

The app is split into a backend route layer, a storage layer, and a template-based frontend. That kept the project easy to reason about and fast to iterate on as a team.

- Flask routes handle uploads, selections, preview, and AI endpoints
- SQLite stores metadata while files are stored on disk.
- Jinja2 templates render the UI and keep the frontend simple.

FRONTEND & UX

The frontend focuses on clarity. Users can see what is selected, what action is happening, and what to do next. The UI is intentionally simple, so the demo stays stable and understandable.

- Home page for upload, preview, selection, and deletion.
- Chat, Flashcards, Quiz, and Code Review pages for AI tasks.
- Loading states, empty states, responsive styling, and validation feedback.

AI FEATURES & LAYER 2

For the AI layer, we connected the selected documents to the generation workflows and exposed prompt controls. For the extra challenge, we chose selective context so the AI can work with more relevant document pieces instead of everything at once.

- Chat, flashcards, quiz, and code review are all corpus-aware.
- Users can adjust tone and generation parameters.
- Layer 2 choice: RAG-style selective context for more focused retrieval.

Testing

TESTING AND RELIABILITY

We focused on making the app predictable under normal and failure cases. That includes persistence checks, validation tests, and edge cases such as empty files and oversized uploads.

- Upload, delete, select, persistence, empty-file, oversized-file, and concurrent-save tests.
- Regression tests stay green after the later Stage 3 polish work.
- The app now reports clear user-facing errors and security headers.

CONCLUSION

To conclude, Corpus Forge is a complete local web application with a clear flow, tested behavior, and a documented architecture.

- A working local web application with persistence and AI workflows.
- A clean team split across backend, frontend, and QA/delivery.
- A project we can explain, demo, and defend.

Thank you, and we are happy to answer questions.